

Paimon Lumina 模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-lumina

分支：release-1.3.1

生成工具：AI Architecture Analyzer

生成日期：2026年03月28日

第1章 模块概述

1.1 模块定位

paimon-lumina 是 Apache Paimon 的向量索引扩展模块，集成了阿里巴巴存储服务团队开发的 Lumina 向量搜索库，为 Paimon 的全局索引框架（Global Index Framework）提供高性能的近似最近邻搜索（ANN）能力。该模块通过 JNI 调用 Lumina 原生库，支持 DiskANN 图索引算法，能够在磁盘上高效构建和搜索大规模向量数据集。

该模块作为 Paimon 全局索引体系的一个 SPI 插件实现，标识符为 lumina-vector-ann，可通过 Java SPI 机制自动发现和加载。模块仅支持 x86_64 架构，目前不支持 ARM（如 Apple Silicon）。

1.2 核心能力

- DiskANN 向量索引构建 — 支持 pretrain + insert + dump 三阶段构建流程
- 高效 ANN 搜索 — 支持 top-K 近似最近邻搜索和带过滤条件的搜索
- 三种距离度量 — L2（欧氏距离）、Cosine（余弦距离）、Inner Product（内积）
- 多种向量编码 — rawf32（原始浮点）、PQ（乘积量化）、SQ8（标量量化）
- 内存友好的写入 — 向量溢写到磁盘临时文件，Java 堆内存固定约 8MB
- 懒加载搜索 — Reader 使用双重检查锁模式（DCL）延迟加载索引
- 完整 I/O 统计 — 分阶段统计字节读取数、seek 次数、读取次数和耗时

1.3 源码文件清单

文件名	行数	职责描述
LuminaVectorGlobalIndexerFactory	41	SPI 工厂类，标识符 lumina-vector-ann
LuminaVectorGlobalIndexer	54	全局索引器，桥接 Writer 和 Reader 创建
LuminaVectorIndexOptions	250	配置选项定义与解析，lumina.* 前缀处理
LuminaVectorMetric	62	向量距离度量枚举（L2/Cosine/InnerProduct）
LuminalIndex	219	Lumina 原生索引操作的 Java 安全封装
LuminalIndexMeta	108	索引元数据序列化/反序列化（JSON Map格式）
LuminaVectorGlobalIndexWriter	448	向量索引写入器，磁盘溢写 + 三阶段构建
LuminaVectorGlobalIndexReader	531	向量索引读取器，DCL 懒加载 + ANN 搜索
LuminaScoredGlobalIndexResult	58	带分数的搜索结果封装（位图+分数映射）

第2章 核心架构设计

2.1 整体分层架构

paimon-lumina

模块采用清晰的四层架构设计：SPI
服务发现层、索引器桥接层、读写器核心层、以及
原生引擎层。各层通过接口解耦，便于扩展和测试。

JNI

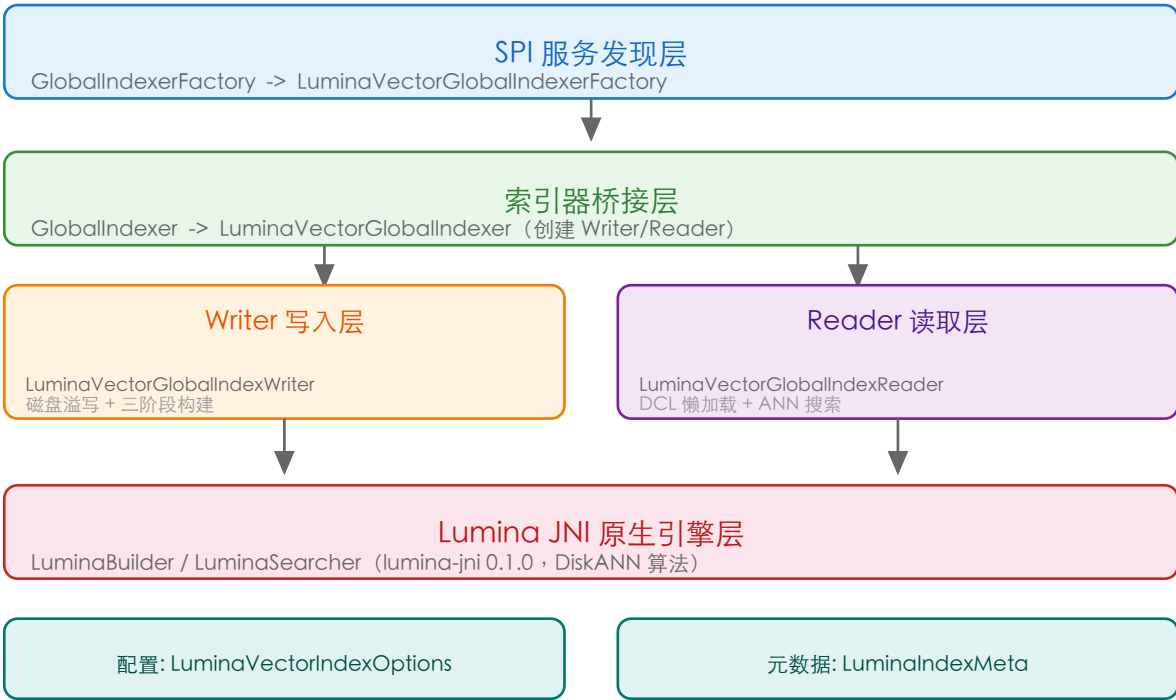


图2-1 paimon-lumina 整体分层架构图

2.2 类关系与继承体系

paimon-lumina

模块的类关系基于

paimon-common

中定义的全局索引接口体系。以下是核心接口与实现类的映射关系：

接口（paimon-common）	实现类（paimon-lumina）	说明
GlobalIndexerFactory	LuminaVectorGlobalIndexerFactory	SPI 工厂，标识符 lumina-vector-ann
GlobalIndexer	LuminaVectorGlobalIndexer	索引器桥接，创建 Writer/Reader
GlobalIndexSingletonWriter	LuminaVectorGlobalIndexWriter	单列向量写入器
GlobalIndexReader	LuminaVectorGlobalIndexReader	向量搜索读取器
ScoredGlobalIndexResult	LuminaScoredGlobalIndexResult	带分数的搜索结果

2.3 Paimon 全局索引概念映射

paimon-lumina 将 Paimon 全局索引框架的概念映射到 Lumina 向量搜索引擎上：

Paimon 概念	Lumina 对应	说明
GlobalIndexFileWriter	PositionOutputStream	文件写入流抽象
GlobalIndexFileReader	SeekableInputStream	文件读取流抽象
GlobalIndexIOMeta	LuminaIndexMeta	索引文件元数据
ResultEntry	dump 输出文件	构建完成后的索引文件描述
VectorSearch	LuminaSearcher.search()	向量近邻搜索请求
RoaringNavigableMap64	long[] labels	搜索结果行 ID 集合

2.4 外部依赖分析

依赖	版本	用途
paimon-common	\${project.version}	全局索引框架接口（provided）
lumina-jni	0.1.0	Lumina 向量搜索 JNI 绑定
paimon-shade-jackson-2	shaded	JSON 序列化（索引元数据）

第3章 核心类设计模式详解

3.1 LuminaVectorGlobalIndexerFactory — 抽象工厂模式 (Abstract Factory)

LuminaVectorGlobalIndexerFactory 实现了 GlobalIndexerFactory 接口，作为 Lumina 向量索引的工厂入口点。通过 Java SPI 机制 (META-INF/services) 注册，使得 Paimon 核心框架可以透明地发现和实例化 Lumina 索引器。

设计意图：将索引器的创建逻辑与使用逻辑分离，遵循开闭原则。新增索引类型只需新增工厂实现。

关键方法：

- identifier() — 返回 "lumina-vector-ann"，用于 SPI 类型匹配
- create(DataField, Options) — 根据字段类型和配置创建 LuminaVectorGlobalIndexer

3.2 LuminaVectorGlobalIndexer — 桥接模式 (Bridge)

LuminaVectorGlobalIndexer 实现了 GlobalIndexer 接口，作为抽象索引操作 (Writer/Reader) 与具体 Lumina 实现之间的桥梁。它将 Paimon 全局索引框架的文件 IO 抽象 (GlobalIndexFileWriter/GlobalIndexFileReader) 与 Lumina 特定实现连接。

设计意图：解耦索引器的抽象层 (创建 Writer/Reader) 与实现层 (Lumina 具体逻辑)，使两者可独立变化。

- createWriter() — 创建 LuminaVectorGlobalIndexWriter 实例
- createReader() — 创建 LuminaVectorGlobalIndexReader 实例

3.3 LuminaIndex — 外观模式 (Facade) + 双模式切换

LuminaIndex 为 Lumina 原生 Builder 和 Searcher 提供统一的 Java 安全封装层。通过两个静态工厂方法 createForBuild() 和 fromStream() 创建不同模式的实例，内部根据模式委派到 LuminaBuilder 或 LuminaSearcher。

设计意图：简化客户端代码，隐藏原生 JNI 对象的复杂性和状态管理。通过 ensureOpen()/ensureBuilder()/ensureSearcher() 提供状态安全保障。

- createForBuild() — 构建模式：pretrain -> insert -> dump
- fromStream() — 搜索模式：search / searchWithFilter
- filterSearchOptions() — 只传递 search.*/diskann.search.* 前缀的选项给原生搜索
- close() — synchronized 线程安全关闭，防止双重关闭

3.4 LuminaVectorGlobalIndexWriter — 策略模式 (Strategy) + 模板方法模式 (Template Method)

Writer 采用策略模式处理不同的向量数据类型输入 (float[] 和 InternalArray)，并使用模板方法模式定义三阶段索引构建流程：pretrain -> insert -> dump。

磁盘溢写策略：向量数据通过 8MB DirectByteBuffer 缓冲后写入临时文件，避免 Java 堆内存随数据集规模线性增长。FileBackedDataset 通过 LuminaDataset 回调接口流式供给。

关键内部类：

- FileBackedDataset — 实现 LuminaDataset 接口，从磁盘临时文件流式读取向量（带进度日志）
- OutputStreamFileOutput — 适配器模式，将 PositionOutputStream 适配为 LuminaFileOutput
- setEnv() — 通过反射设置 JVM 环境变量（ConcurrentHashMap + DCL），线程安全

3.5 LuminaVectorGlobalIndexReader — 访问者模式 (Visitor) + 懒加载 (Lazy Initialization)

Reader 实现了 GlobalIndexReader 接口（继承自 FunctionVisitor），采用访问者模式处理不同类型的查询谓词。仅支持 visitVectorSearch()，其余所有谓词方法（Equal/GreaterThan/Like 等 14 种）均返回 Optional.empty()。

双重检查锁 (DCL) 懒加载：ensureLoaded() 方法使用 volatile + synchronized 实现线程安全的延迟索引加载。

距离到分数转换策略：

- L2: $score = 1 / (1 + distance)$ — 距离越小分数越高
- Cosine: $score = 1 - distance$ — 距离越小分数越高
- Inner Product: $score = distance$ — 内积本身即为相似度

关键内部类：

- InputStreamFileInput — 适配器模式，将 SeekableInputStream 适配为 LuminaFileInput JNI API，并提供 I/O 统计
- ScoredRow — 内部 POJO，用于 PriorityQueue 最小堆维护 top-K 结果

3.6 LuminaVectorIndexOptions — 建造者模式 (Builder) + 校验链

Options 类通过 ConfigOption 定义 12 个配置项，在构造时执行一系列校验和转换：正值校验、PQ+Cosine 组合禁止校验、PQ M 维度上限自动封顶。toLuminaOptions() 方法将 lumina.* 前缀剥离后构建原生 Lumina 选项 Map。

- buildLuminaOptions() — 先填充所有已知选项默认值，再覆盖用户自定义 lumina.* 选项
- capPqM() — 确保 PQ 子量化器数不超过向量维度
- validateEncodingMetricCombination() — 禁止 PQ + cosine 组合

第4章 核心流程图

4.1 向量索引构建流程

索引构建由 `LuminaVectorGlobalIndexWriter` 驱动，分为数据收集和索引构建两大阶段。数据收集阶段将向量溢写到磁盘临时文件；索引构建阶段执行 `pretrain` -> `insert` -> `dump` 三步操作。

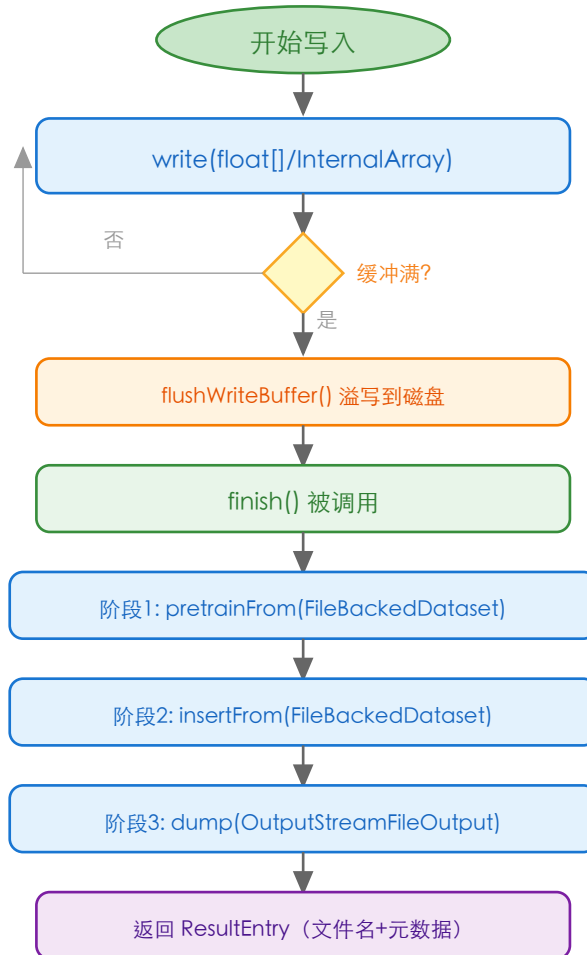


图4-1 向量索引构建流程图

4.2 向量搜索流程

向量搜索由 `LuminaVectorGlobalIndexReader` 驱动，采用懒加载模式。首次搜索时通过 DCL 机制加载索引文件到 `LuminaSearcher`，后续搜索直接复用。

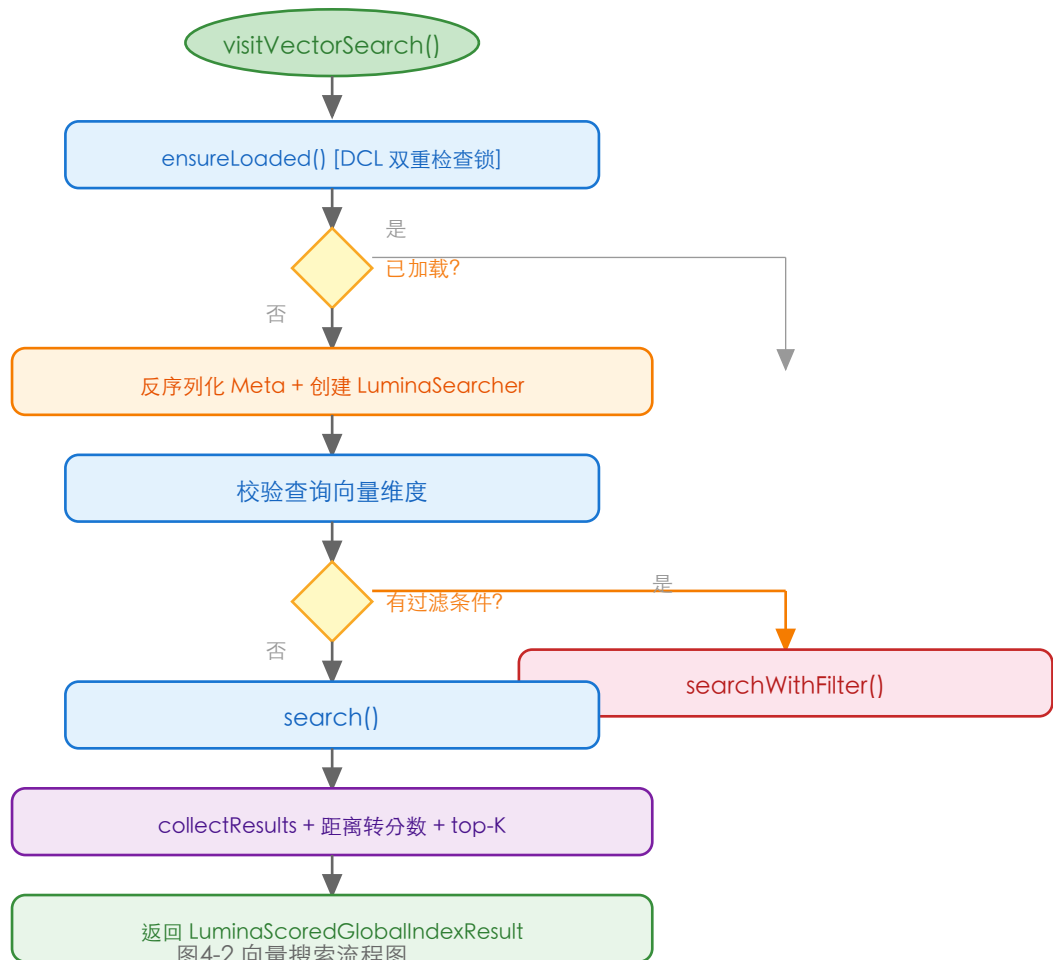


图4-2 向量搜索流程图

4.3 SPI 服务发现与实例化流程

Paimon 核心框架通过 Java SPI 机制发现和实例化 Lumina 向量索引器。服务注册文件 `META-INF/services/org.apache.paimon.globalindex.GloballIndexerFactory` 声明了 `LuminaVectorGlobalIndexerFactory` 实现类。

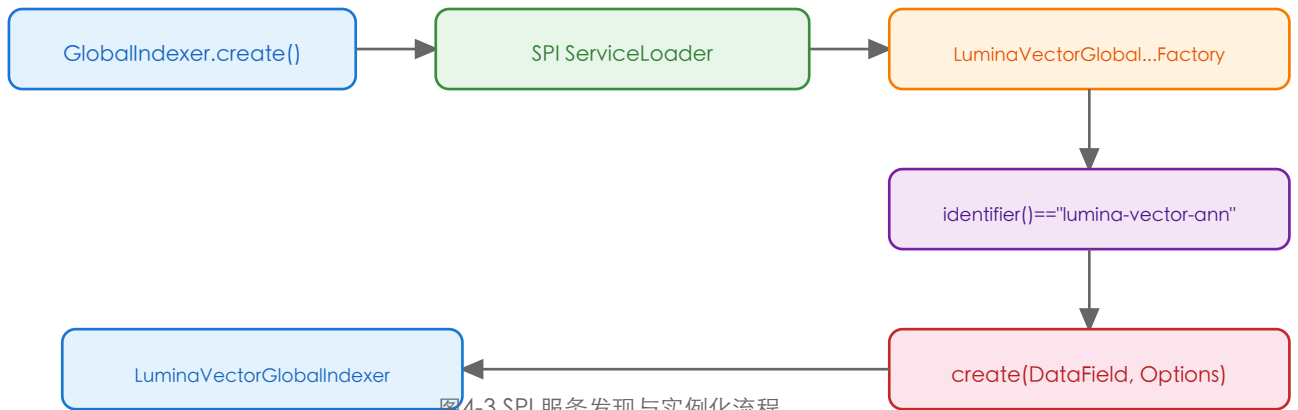


图4-3 SPI 服务发现与实例化流程

第5章 核心时序图

5.1 索引构建时序图

以下时序图展示了从外部调用者发起索引构建到最终生成索引文件的完整交互过程。核心参与者包括：调用者（DefaultGlobalIndexBuilder）、Writer、临时文件、LuminaIndex 和 LuminaBuilder。

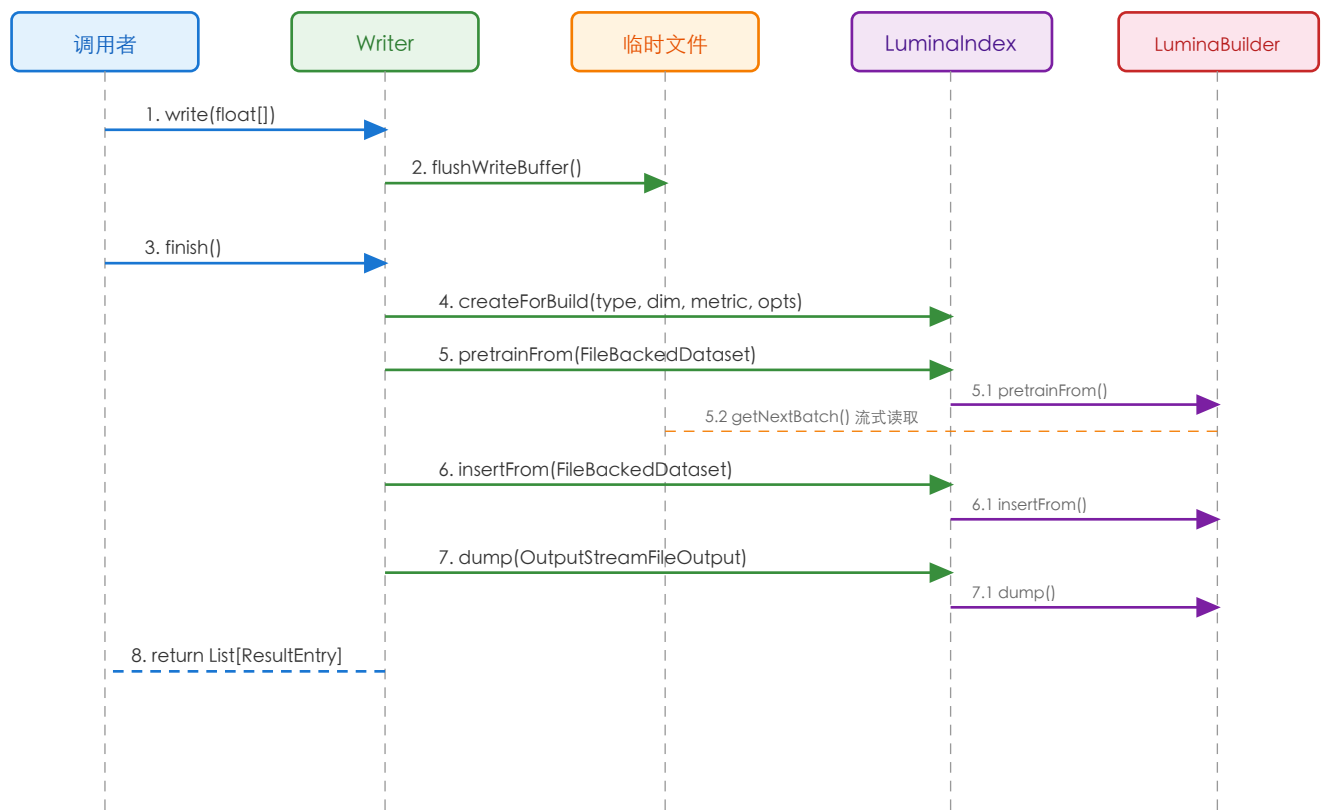


图5-1 索引构建完整时序图

5.2 向量搜索时序图

以下时序图展示了向量搜索的完整交互过程。核心参与者包括：查询引擎、Reader、LuminalIndex、LuminaSearcher 和底层 I/O。

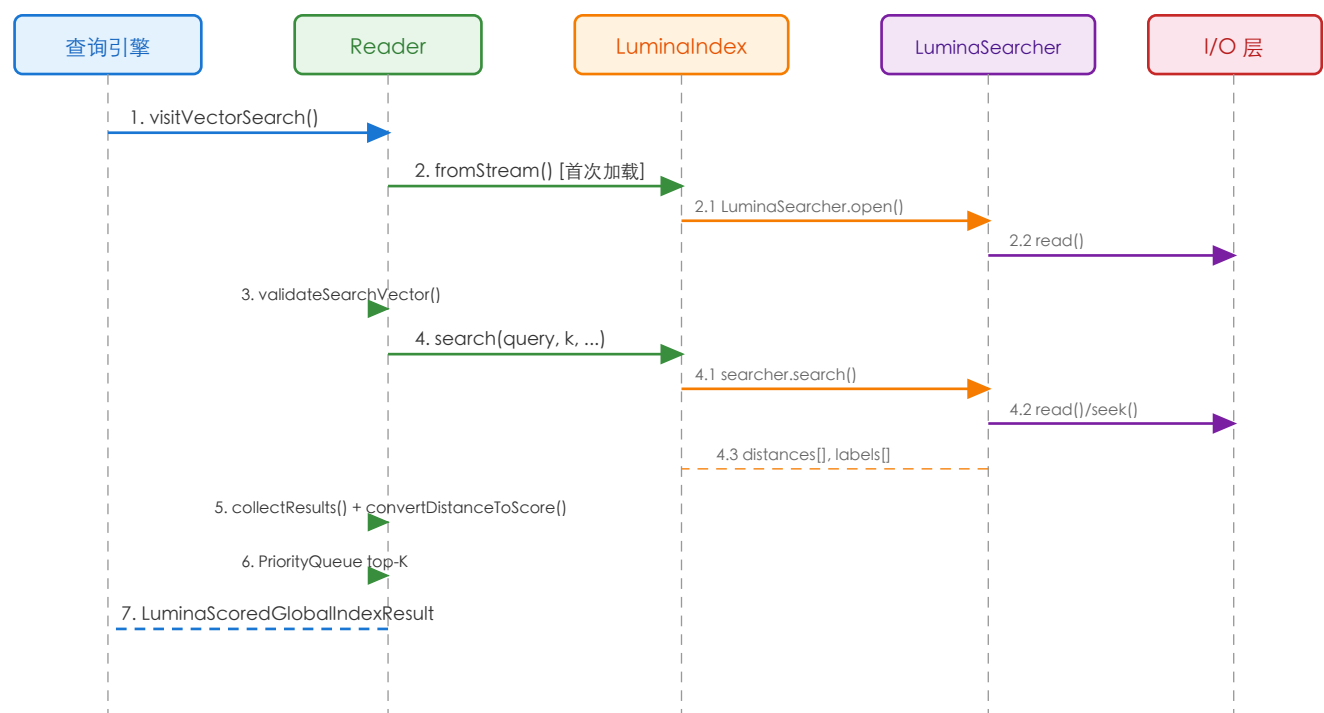


图5-2 向量搜索完整时序图

第6章 关键场景调用链

6.1 场景一：SPI 发现与索引器创建

触发条件：Paimon 核心框架需要为某个字段创建向量全局索引时。

调用链路：

步骤	类.方法	说明
1	GlobalIndexer.create(type, dataField, options)	静态工厂方法入口
2	GlobalIndexerFactoryUtils.load("lumina-vector-ann")	SPI ServiceLoader 加载工厂
3	LuminaVectorGlobalIndexerFactory.create(DataField, Options)	创建 Indexer 实例
4	new LuminaVectorGlobalIndexer(fieldType, options)	初始化选项解析
5	new LuminaVectorIndexOptions(options)	校验维度/度量/编码组合

6.2 场景二：大规模向量索引构建

触发条件：对包含向量列的表执行全局索引构建操作。

调用链路：

步骤	类.方法	说明
1	LuminaVectorGlobalIndexer.createWriter(fileWriter)	创建 Writer
2	LuminaVectorGlobalIndexWriter.<init>()	创建临时文件 + 分配 8MB DirectByteBuffer
3	Writer.write(float[]) [循环 N 次]	向量写入缓冲，满时 flushWriteBuffer() 溢写到磁盘
4	Writer.finish()	触发索引构建
5	configureExecutorThreadCount()	反射设置 LUMINA_EXECUTOR_THREAD_COUNT 环境变量
6	LuminalIndex.createForBuild(type, dim, metric, opts)	创建 LuminaBuilder
7	LuminalIndex.pretrainFrom(FileBackedDataset)	从临时文件流式预训练
8	LuminalIndex.insertFrom(FileBackedDataset)	从临时文件流式插入
9	LuminalIndex.dump(OutputStreamFileOutput)	序列化索引到全局索引文件

步骤	类.方法	说明
10	<code>new ResultEntry(fileName, count, meta.serialize())</code>	返回构建结果

6.3 场景三：向量相似性搜索（无过滤）

触发条件：用户通过 `VectorSearchBuilder` 发起向量搜索查询。

调用链路：

步骤	类.方法	说明
1	<code>LuminaVectorGlobalIndexer.createReader(fileReader, files)</code>	创建 Reader
2	<code>Reader.visitVectorSearch(VectorSearch)</code>	入口方法
3	<code>Reader.ensureLoaded()</code> [DCL]	首次加载：反序列化 Meta + 创建 LuminaSearcher
4	<code>LuminalIndex.fromStream(type, fileInput, size, dim, metric, opts)</code>	打开原生搜索索引
5	<code>Reader.validateSearchVector(query)</code>	校验查询向量维度匹配
6	<code>LuminalIndex.search(query, 1, k, distances, labels, searchOpts)</code>	执行原生 ANN 搜索
7	<code>Reader.collectResults(distances, labels, k, k, topK, metric)</code>	PriorityQueue 收集 top-K
8	<code>convertDistanceToScore(distance, metric)</code>	距离转分数（L2/Cosine/IP）
9	<code>new LuminaScoredGlobalIndexResult(bitmap, id2scores)</code>	封装结果

6.4 场景四：带过滤条件的向量搜索

触发条件：查询携带 `includeRowIds` 位图进行范围限定的向量搜索。

调用链路：

步骤	类.方法	说明
1	<code>VectorSearch.withIncludeRowIds(RoaringNavigableMap64)</code>	设置行 ID 过滤条件
2	<code>Reader.visitVectorSearch(vectorSearch)</code>	入口
3	<code>includeRowIds.iterator() -> long[] scopedIds</code>	将位图转为 long 数组

步骤	类.方法	说明
4	<code>effectiveK = min(topK, scopedIds.length)</code>	限制搜索结果数不超过候选数
5	<code>searchOptions.put("search.thread_safe_filter", "true")</code>	启用线程安全过滤
6	<code>LuminalIndex.searchWithFilter(query, 1, k, dist, labels, scopedIds, opts)</code>	原生过滤搜索
7	<code>Reader.collectResults() + convertDistanceToScore()</code>	收集并转换结果

6.5 场景五：端到端集成扫描

触发条件：Paimon 表扫描中触发向量索引过滤（VectorSearchBuilder）。

调用链路：

步骤	类.方法	说明
1	table.newVectorSearchBuilder()	创建向量搜索构建器
2	builder.withVector(query).withLimit(k).withVectorColumn(col)	设置搜索参数
3	builder.newVectorScan().scan()	创建向量扫描并执行
4	builder.newVectorRead().read(scanResult)	读取向量索引
5	Reader.visitVectorSearch() [内部调用]	执行 Lumina ANN 搜索
6	readBuilder.newScan().withGlobalIndexResult(result)	将索引结果注入表扫描
7	readBuilder.newRead().createReader(plan)	创建数据读取器
8	reader.forEachRemaining(row -> ...)	遍历过滤后的行数据

6.6 场景六：索引元数据序列化与兼容

触发条件：索引构建完成时序列化元数据，或搜索时反序列化元数据。

调用链路：

步骤	类.方法	说明
1	Writer.buildIndex() -> new LuminalIndexMeta(options.toLuminaOptions())	创建元数据对象
2	LuminalIndexMeta.serialize()	Jackson 序列化为 JSON Map (Map<String,String>)
3	new ResultEntry(fileName, count, metaBytes)	元数据附加到结果条目
4	Reader.ensureLoaded() -> LuminalIndexMeta.deserialize(ioMeta.metadata())	反序列化元数据
5	indexMeta.dim() / indexMeta.metric() / indexMeta.indexType()	提取维度、度量、索引类型
6	searcherOptions.putAll(indexMeta.options())	元数据选项覆盖默认选项

第7章 配置与扩展

7.1 配置属性

所有配置项使用 `lumina.*` 前缀，作为 Paimon 表属性设置。模块在初始化时自动剥离前缀传给原生 Lumina 引擎。

配置项	类型	默认值	说明
<code>lumina.index.dimension</code>	<code>int</code>	128	向量维度
<code>lumina.index.type</code>	<code>string</code>	<code>diskann</code>	索引类型
<code>lumina.distance.metric</code>	<code>string</code>	<code>inner_product</code>	距离度量 (<code>l2/cosine/inner_product</code>)
<code>lumina.encoding.type</code>	<code>string</code>	<code>pq</code>	编码类型 (<code>rawf32/sq8/pq</code>)
<code>lumina.pretrain.sample_ratio</code>	<code>double</code>	0.2	预训练采样比例
<code>lumina.diskann.build.ef_construction</code>	<code>int</code>	1024	图构建动态候选列表大小
<code>lumina.diskann.build.neighbor_count</code>	<code>int</code>	64	每节点最大邻居数
<code>lumina.diskann.build.thread_count</code>	<code>int</code>	32	构建线程数
<code>lumina.diskann.search.list_size</code>	<code>int</code>	1.5x topK	搜索列表大小（自动设置）
<code>lumina.diskann.search.beam_width</code>	<code>int</code>	4	搜索束宽
<code>lumina.encoding.pq.m</code>	<code>int</code>	64	PQ 子量化器数（自动封顶到维度）
<code>lumina.search.parallel_number</code>	<code>int</code>	5	搜索并行度

7.2 扩展点

- 新索引类型： `LuminalIndex.createForBuild()` 和 `fromStream()` 已支持通过 `indexType` 参数选择不同算法，目前仅实现 DiskANN
- 新编码方式：通过 `lumina.encoding.type` 配置项，可透传给原生 Lumina 引擎
- 自定义搜索选项：任何以 `lumina.` 为前缀的表属性都会自动传递给原生引擎
- 新存储后端：通过 Paimon FileIO 抽象，已支持 Local/OSS/PVFS 等存储

7.3 约束与限制

- 架构限制：仅支持 x86_64，不支持 ARM/Apple Silicon
- 编码限制：PQ 编码不支持 cosine 度量
- 分片限制：每个 Reader 只支持恰好 1 个索引文件 (`checkArgument(ioMetas.size() == 1)`)
- 数据类型：仅支持 float 数组类型 (`ArrayType(FloatType)`)

第8章 设计亮点与总结

8.1 设计亮点

- 磁盘溢写架构：Writer 通过 8MB DirectByteBuffer + 临时文件实现内存恒定，无论数据集多大，Java 堆内存固定在约 8MB。FileBackedDataset 通过 LuminaDataset 回调接口实现零拷贝流式供给。
- 双模式 LuminalIndex：通过静态工厂方法实现构建模式和搜索模式的清晰分离，配合 ensureBuilder()/ensureSearcher() 状态检查，防止误用。close() 使用 synchronized 双重检查确保线程安全。
- DCL 懒加载搜索：Reader 的 ensureLoaded() 使用 volatile + synchronized 双重检查锁，首次搜索时才加载索引文件，后续搜索零开销复用。
- 精细 I/O 统计：InputStreamFileInput 将 I/O 统计分为 open 阶段和 search 阶段，便于性能调优和问题诊断。统计指标包括字节读取数、seek 次数、read 次数及耗时。
- paimon-cpp 元数据兼容：LuminalIndexMeta 使用 Map<String, String> JSON 格式，与 paimon-cpp 的元数据格式完全兼容，实现跨语言索引互操作。
- 线程安全环境变量：setEnv() 方法通过 ConcurrentHashMap + synchronized 双重检查锁 + 反射，实现多分片并行构建时的线程安全环境变量设置。

8.2 设计限制

- 平台限制：Lumina 原生库仅支持 x86_64 架构，无法在 Apple Silicon 等 ARM 平台上运行
- 单分片限制：Reader 强制要求每个分片恰好 1 个索引文件，不支持多文件合并搜索
- 环境变量全局性：LUMINA_EXECUTOR_THREAD_COUNT 通过反射设置环境变量影响整个 JVM，不支持 per-instance 线程池配置
- 反射依赖：setEnv() 依赖反射修改 System.getenv() 内部 Map，在 Java 17+ 的强封装模式下可能失败

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	四层清晰架构，接口与实现完全分离，SPI 可插拔
设计模式运用	★★★★★	融合工厂、桥接、外观、策略、访问者、适配器等多种模式
内存管理	★★★★★	磁盘溢写 + 固定缓冲，堆内存恒定约 8MB
线程安全	★★★★☆	DCL 懒加载、synchronized 关闭、但 Writer 本身非线程安全
可扩展性	★★★★☆	SPI 插件化 + 配置驱动，但受限于单分片设计
跨语言兼容	★★★★★	元数据格式与 paimon-cpp 完全兼容
测试覆盖	★★★★★	涵盖度量/维度/过滤/异常/大规模/元数据覆盖等场景

评估维度	评分	说明
性能基准	★★★★☆	完善的 Benchmark，支持 Local/OSS/PVFS 多存储